

Análise comparativa entre as seleções de pivô aleatório, posição central, mediana e último na implementação do algoritmo de ordenação *Quick Sort* para diferentes entradas

Fabio K. Sato¹, Thalita W. Pereira², Vinicius M. Marcos³

¹Universidade Estadual do Oeste do Paraná (UNIOESTE)
Caixa Postal 711 – 85.819-110 – Cascavel – PR – Brazil

{fabio.sato, thalita.pereira1, vinicius.marcos1}@unioeste.br

Abstract. *This paper presents a comparative study of four pivot selection strategies for the QuickSort algorithm: random, central, randomized median of three, and the last element. The objective is to corroborate theoretical concepts regarding the algorithm's execution time by illustrating its behavior across different data scenarios. Through asymptotic cost analysis and empirical execution time measurements on arrays up to two million elements, the results highlight how structured data drastically degrades the performance of the naive last-element pivot approach to $\mathcal{O}(n^2)$, whereas the central, random, and randomized median of three strategies maintain efficient $\mathcal{O}(n \log n)$ execution.*

Resumo. *Este artigo apresenta um estudo comparativo entre quatro estratégias de seleção de pivô para o algoritmo QuickSort: aleatório, central, mediana de três aleatorizada e o último elemento. O objetivo é corroborar os conceitos teóricos relacionados ao tempo de execução do algoritmo, ilustrando o comportamento das estratégias em diferentes cenários de dados. Através da análise teórica de custo assintótico e medições empíricas de tempo de execução em vetores de até dois milhões de elementos, os resultados demonstram como dados estruturados degradam severamente o desempenho da abordagem do pivô no último elemento para $\mathcal{O}(n^2)$, enquanto as estratégias central, aleatória e mediana de três aleatorizada se mantêm eficientes em $\mathcal{O}(n \log n)$.*

1. Introdução

Desenvolvido em 1960 por Charles Antony Richard Hoare e publicado em 1962 [Hoare 1962], o QuickSort é um algoritmo de ordenação que utiliza o paradigma de *Dividir para Conquistar*. Ele atua dividindo o vetor em vetores menores e os ordenando de forma independente, mantendo os números menores à esquerda e os maiores à direita do pivô [Cormen et al. 2009, Sedgewick and Wayne 2011].

A principal característica do QuickSort é a sua função de particionamento [Cormen et al. 2009]. O algoritmo começa com a escolha de um pivô e reorganiza os elementos, garantindo que, ao final do particionamento, o pivô esteja em sua posição final ordenada [Cormen et al. 2009]. Seu pior caso possui tempo de execução $\mathcal{O}(n^2)$, mas, apesar disso, o QuickSort é geralmente muito eficiente na prática, possuindo um tempo de execução esperado de $\mathcal{O}(n \log n)$ [Knuth 1998].

O objetivo deste artigo é aplicar e corroborar conceitos adquiridos com relação a este método de ordenação. Para isso, o presente trabalho ilustra o comportamento do

algoritmo a partir da implementação de quatro estratégias distintas de escolha de pivô: Aleatório, Central, Mediana de Três Aleatorizada e Último elemento. A análise do comportamento do algoritmo frente a diferentes configurações de dados (aleatórios, crescentes, decrescentes e parcialmente ordenados) fornece uma base sólida para a compreensão empírica e teórica da eficiência computacional de cada técnica.

O artigo está organizado da seguinte forma: a Seção 2 aborda as implementações e a análise teórica do custo assintótico; a Seção 3 detalha a configuração da máquina, o ambiente do experimento e a metodologia de testes; a Seção 4 apresenta os gráficos com a discussão analítica sobre os comportamentos do algoritmo; e a Seção 5 expõe as considerações finais.

2. Desenvolvimento e Fundamentação Teórica

A implementação foi unificada sob uma mesma linguagem de programação, adotando soluções idênticas para todos os métodos, diferenciando-se estritamente na escolha do elemento pivô. Conforme definido, foram elaboradas as seguintes estratégias:

1. **QuickSort Randômico:** O pivô é escolhido sorteando-se um índice aleatório dentro do intervalo do subvetor. Esta aleatoriedade reduz a probabilidade de particionamentos desbalanceados em qualquer configuração específica de dados [Cormen et al. 2009].
2. **QuickSort Central:** O pivô é determinado selecionando-se o elemento na posição central do vetor. Essa estratégia reduz a chance de o pivô ser um valor extremo (como o maior ou menor do vetor) em dados já ordenados [Cormen et al. 2009].
3. **QuickSort Mediana de Três Aleatorizada:** Diferentemente da variante clássica descrita em [Cormen et al. 2009, Sedgewick and Wayne 2011], que amostra três posições fixas (primeira, central e última do subvetor), a estratégia implementada realiza o sorteio aleatório de três posições distintas no intervalo do subvetor. O pivô escolhido é o valor correspondente à mediana estatística entre as três chaves sorteadas. Essa variante aleatorizada também mitiga o risco de pior caso sistemático.
4. **QuickSort Último:** Por fim, o pivô adotado é fixamente o último elemento do subvetor atual. Essa é uma das abordagens mais clássicas, porém sabidamente vulnerável a distribuições não uniformes de dados [Knuth 1998].

2.1. Detalhes de Implementação

Abaixo apresentamos o trecho de código responsável pelo particionamento utilizando a estratégia do pivô central. Nela, o elemento particionador é selecionado a partir do cálculo aritmético direto da posição intermediária do subvetor.

```
1 void QuickSortCentral(int *vet, int esq, int dir)
2 {
3     while (esq < dir)
4     {
5         int i = esq, j = dir;
6         int pivo = vet[(esq + dir) / 2];
7
8         while (i <= j)
9         {
10             while (vet[i] < pivo && i < dir) i++;
```

```

11         while (vet[j] > pivo && j > esq) j--;
12
13         if (i <= j)
14         {
15             trocar(&vet[i], &vet[j]);
16             i++;
17             j--;
18         }
19     }
20
21     if ((j - esq) < (dir - i))
22     {
23         if (j > esq) QuickSortCentral(vet, esq, j);
24         esq = i;
25     }
26     else
27     {
28         if (i < dir) QuickSortCentral(vet, i, dir);
29         dir = j;
30     }
31 }
32 }

```

Código 1. Função *Quicksort* com Pivô Central.

A segunda abordagem utiliza a função `rand()` associada à aritmética modular para sortear o índice do pivô, conforme ilustra o Código 2.

```

1 void QuickSortRandomico(int *vet, int esq, int dir)
2 {
3     while (esq < dir)
4     {
5         int i = esq, j = dir;
6
7         int indice_pivo = esq + rand() % (dir - esq + 1);
8         int pivo = vet[indice_pivo];
9
10        while (i <= j)
11        {
12            while (vet[i] < pivo && i < dir) i++;
13            while (vet[j] > pivo && j > esq) j--;
14
15            if (i <= j)
16            {
17                trocar(&vet[i], &vet[j]);
18                i++;
19                j--;
20            }
21        }
22
23        if ((j - esq) < (dir - i))
24        {
25            if (j > esq) QuickSortRandomico(vet, esq, j);
26            esq = i;
27        }
28        else
29        {

```

```

30         if (i < dir) QuickSortRandomico(vet, i, dir);
31         dir = j;
32     }
33 }
34 }

```

Código 2. Função Quicksort com Pivô Aleatório.

Já o Código 3 seleciona três índices aleatórios no intervalo do subvetor. A mediana algébrica entre esses três valores é então identificada por meio de condicionais e designada como pivô.

```

1 void QuickSortMediana3(int *vet, int esq, int dir)
2 {
3     while (esq < dir)
4     {
5         int i = esq, j = dir;
6
7         // Sorteia tres posicoes aleatorias no intervalo atual
8         int idx1 = esq + rand() % (dir - esq + 1);
9         int idx2 = esq + rand() % (dir - esq + 1);
10        int idx3 = esq + rand() % (dir - esq + 1);
11
12        int v1 = vet[idx1], v2 = vet[idx2], v3 = vet[idx3];
13        int pivo;
14
15        // Identifica a mediana dos tres valores sorteados
16        if ((v1 >= v2 && v1 <= v3) || (v1 <= v2 && v1 >= v3)) pivo = v1
17        ;
18        else if ((v2 >= v1 && v2 <= v3) || (v2 <= v1 && v2 >= v3)) pivo
19        = v2;
20        else pivo = v3;
21
22        while (i <= j)
23        {
24            while (vet[i] < pivo && i < dir) i++;
25            while (vet[j] > pivo && j > esq) j--;
26
27            if (i <= j)
28            {
29                trocar(&vet[i], &vet[j]);
30                i++;
31                j--;
32            }
33        }
34
35        if ((j - esq) < (dir - i))
36        {
37            if (j > esq) QuickSortMediana3(vet, esq, j);
38            esq = i;
39        }
40        else
41        {
42            if (i < dir) QuickSortMediana3(vet, i, dir);
43            dir = j;
44        }
45    }
46 }

```

```
43     }
44 }
```

Código 3. Função *Quicksort* com Pivô selecionado a partir da mediana de três números sorteados aleatoriamente.

Por fim, no algoritmo do Código 4, é realizada a seleção do elemento da extremidade direita do subvetor. Vale ressaltar um detalhe técnico importante: a estrutura de laços aninhados para a movimentação de ponteiros (i e j), bem como a otimização de recursividade em cauda (que prioriza a chamada recursiva sempre para o menor subvetor), foram mantidas rigorosamente idênticas em todos os códigos. Essa técnica, descrita em [Sedgewick and Wayne 2011], limita a profundidade máxima da pilha de chamadas a $\mathcal{O}(\log n)$ mesmo quando o particionamento degenera, evitando estouros de pilha em cenários de pior caso [Cormen et al. 2009]. Isso garante que as diferenças de tempo aferidas na seção de resultados reflitam pura e exclusivamente o impacto de custo gerado pela seleção do pivô.

```
1 void QuickSortUltimo(int *vet, int esq, int dir)
2 {
3     while (esq < dir)
4     {
5         int i = esq, j = dir;
6         int pivo = vet[dir];
7
8         while (i <= j)
9         {
10            while (vet[i] < pivo && i < dir) i++;
11            while (vet[j] > pivo && j > esq) j--;
12
13            if (i <= j)
14            {
15                trocar(&vet[i], &vet[j]);
16                i++;
17                j--;
18            }
19        }
20
21        if ((j - esq) < (dir - i))
22        {
23            if (j > esq) QuickSortUltimo(vet, esq, j);
24            esq = i;
25        }
26        else
27        {
28            if (i < dir) QuickSortUltimo(vet, i, dir);
29            dir = j;
30        }
31    }
32 }
```

Código 4. Função *Quicksort* com Pivô na última posição.

2.2. Análise do Custo Assintótico

A análise da complexidade assintótica é fundamental para avaliar o comportamento de um algoritmo à medida que o tamanho da entrada cresce [Cormen et al. 2009].

Para o caso médio, onde espera-se que a escolha do pivô (seja por aleatoriedade, mediana ou posição central) consiga dividir o arranjo em duas subpartições de tamanhos aproximadamente proporcionais [Cormen et al. 2009], a relação de recorrência baseia-se na divisão sucessiva:

$$T(n) = 2T\left(\frac{n}{2}\right) + \mathcal{O}(n) \quad (1)$$

Aplicando o Teorema Mestre [Cormen et al. 2009], concluímos que o tempo de execução no melhor caso e no caso médio é:

$$T(n) = \mathcal{O}(n \log n) \quad (2)$$

As estratégias Randômica, Central e Mediana de Três Aleatorizada tendem fortemente a este caso médio. Contudo, a estratégia **Último** está sujeita a divisões desbalanceadas crônicas. Ao deparar-se com um vetor já ordenado (crescente) ou reversamente ordenado (decrecente), a escolha do último elemento fará com que o algoritmo gere uma partição com 0 elementos e outra com $n-1$ elementos. Neste pior caso, o particionamento isola um único elemento por vez, gerando a recorrência [Knuth 1998]:

$$T(n) = T(n-1) + \mathcal{O}(n) \quad (3)$$

A resolução dessa recorrência resulta em uma soma aritmética, culminando no pior cenário possível de complexidade [Cormen et al. 2009, Knuth 1998]:

$$T(n) = \mathcal{O}(n^2) \quad (4)$$

3. Metodologia e Detalhamento do Experimento Empírico

A coleta e determinação dos custos empíricos baseou-se no registro estrito do tempo cronológico para realizar a ordenação dos dados, não sendo contabilizados custos paralelos como o tempo de abertura ou formatação dos arquivos em disco.

3.1. Configuração do Ambiente de Testes

Para garantir a reprodutibilidade, os algoritmos foram escritos em C utilizando a IDE **Visual Studio Code**. O ambiente de execução foi montado sobre o Sistema Operacional **Windows 11 Home**, rodando em uma máquina equipada com processador **Intel Core i5-1335U** com **8 GB** de memória RAM.

3.2. Cenário de Entrada e Obtenção de Métricas

Os testes foram conduzidos sob quatro instâncias distintas de conjuntos de dados: vetores Aleatórios, Decrescentes, Crescentes (Ordenados) e Parcialmente Ordenados. O volume (tamanho) de entradas para cada teste progrediu em degraus estruturados, começando com um tamanho de 100 (cem) e chegando no máximo de 2.000.000 (dois milhões) de posições inteiras.

Em relação à coleta de dados, o protocolo de medição empírica seguiu rigorosamente os passos estipulados. Para cada combinação de algoritmo, tipo de vetor e

tamanho de entrada, foram executadas 6 repetições consecutivas e independentes. A primeira execução de cada série foi descartada, neutralizando distorções decorrentes de pré-aquecimento de cache da CPU (*cache warm-up*), faltas de página de memória (*page faults*) e carregamento inicial de registradores. O tempo de execução de cada rodada foi medido em microssegundos utilizando a função `gettimeofday()` e convertido para milissegundos (*ms*). Por fim, o custo empírico final atribuído a cada ponto experimental corresponde à média aritmética simples das 5 execuções restantes.

O gerador de números aleatórios foi inicializado uma única vez, ao início do programa, por meio de `srand(time(NULL))`, de modo que a semente fosse derivada do relógio do sistema e as sequências de sorteio variassem entre execuções. Os vetores de entrada foram lidos a partir de arquivos pré-gerados, uma única vez por combinação de distribuição e tamanho, e mantidos constantes ao longo das 6 repetições de cada série, sendo que apenas cópias independentes do mesmo vetor eram fornecidas a cada ordenação, isolando assim o tempo da ordenação em si. Vale registrar, como limitação metodológica, que a função `rand()` da biblioteca padrão implementa um gerador linear congruente de qualidade modesta, cujo uso do operador módulo introduz viés de uniformidade [Cormen et al. 2009]; para os fins comparativos deste estudo, esse viés afeta de maneira idêntica todas as estratégias, preservando a validade da comparação relativa.

4. Resultados e Análise Comportamental

Nesta seção, os gráficos resultantes das medições são convocados e analisados criticamente, expondo o motivo pelo qual certos algoritmos brilham ou falham dependendo do dado que recebem.

4.1. Desempenho frente a Conjuntos Aleatórios

Iniciando pelo cenário mais favorável, a Figura 1 ilustra a variação do tempo de execução sobre conjuntos onde não há ordenação lógica prévia (arquivos de dados randômicos).

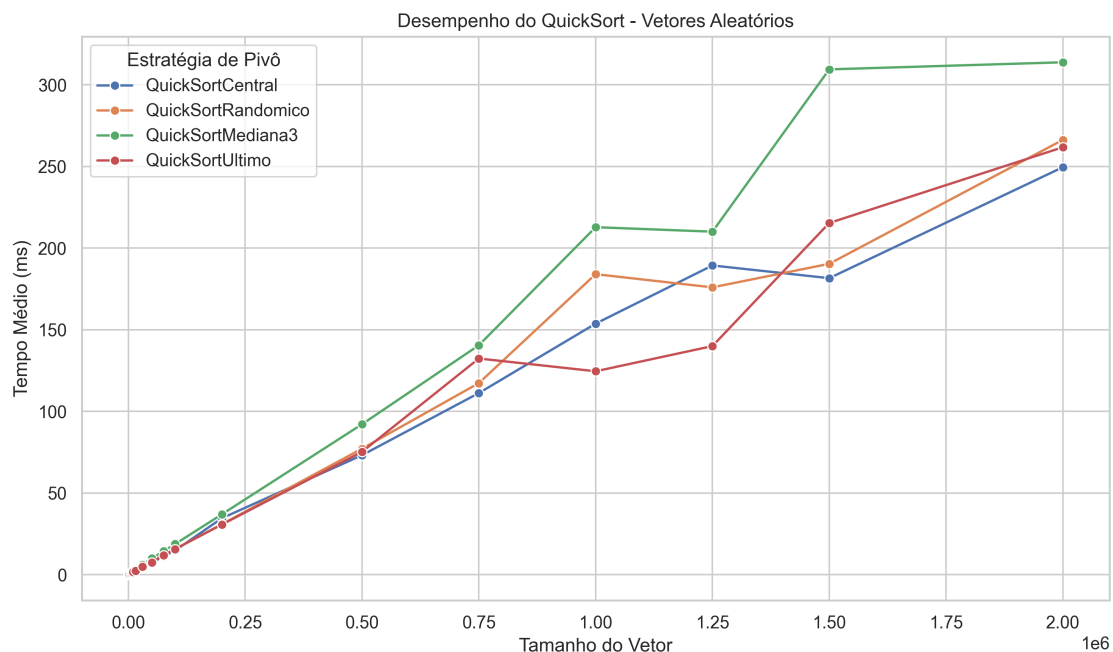


Figura 1. Desempenho do QuickSort no particionamento de vetores aleatórios.

Para complementar a visualização gráfica e permitir uma análise minuciosa, a Tabela 1 detalha os valores cronológicos exatos (em milissegundos) aferidos durante os testes para cada estratégia e tamanho de entrada.

Tabela 1. Tempo médio de execução (em ms) para vetores Aleatórios

Tamanho	QS Central	QS Mediana 3	QS Randômico	QS Último
100	0.0032	0.0176	0.0174	0.0046
200	0.0074	0.0340	0.0246	0.0082
500	0.0284	0.0882	0.0592	0.0282
1.000	0.0738	0.2168	0.1622	0.0912
2.000	0.2942	0.4656	0.2958	0.4712
5.000	1.1132	1.2468	1.3354	0.7576
7.500	1.5622	1.6440	1.3902	1.1482
10.000	1.2848	1.9914	1.7948	1.5804
15.000	2.2692	3.0756	2.5526	2.3086
30.000	4.5056	6.1796	4.9746	4.9796
50.000	7.5124	9.9482	8.0938	7.4708
75.000	11.1004	14.3566	12.1764	11.7760
100.000	14.8096	18.7536	15.7044	15.6812
200.000	34.5388	36.9318	30.8916	30.6740
500.000	73.1648	92.1108	77.1420	75.2140
750.000	111.1836	140.4606	117.2798	132.3474
1.000.000	153.7132	212.7542	183.9944	124.5352
1.250.000	189.3288	210.0118	175.9240	139.9580
1.500.000	181.5476	309.3970	190.3272	215.3314
2.000.000	249.4964	313.7402	266.2166	261.7028

Observando a Figura 1, todos os métodos demonstram um comportamento linear ascendente bastante aglomerado, caracterizando a curva de $\mathcal{O}(n \log n)$. Como o vetor não possui padrão nenhum, mesmo a ingênua escolha do Último elemento atua bem. A estratégia Mediana de Três Aleatorizada apresentou custos ligeiramente superiores aos demais, superando os 300 ms no vetor de 2.000.000, ao passo que os outros alcançaram próximo a 250 ms. Esse sobrecusto se justifica pelo *overhead* adicional decorrente das três chamadas de sorteio aleatório e das comparações condicionais necessárias para determinar a mediana das três chaves a cada partição [Sedgewick and Wayne 2011]. Esse sobrecusto, no entanto, é o preço pago por uma amostragem mais robusta: ao escolher o pivô a partir da mediana de três amostras, a probabilidade de selecionar um elemento extremo da partição cai de forma significativa em relação ao sorteio de um único índice [Cormen et al. 2009].

4.2. Desempenho perante Dados Ordenados (Crescentes)

Em seguida, ao analisar o cenário de vetores previamente ordenados, o efeito destrutivo do pior caso torna-se evidente, conforme ilustrado na Figura 2.

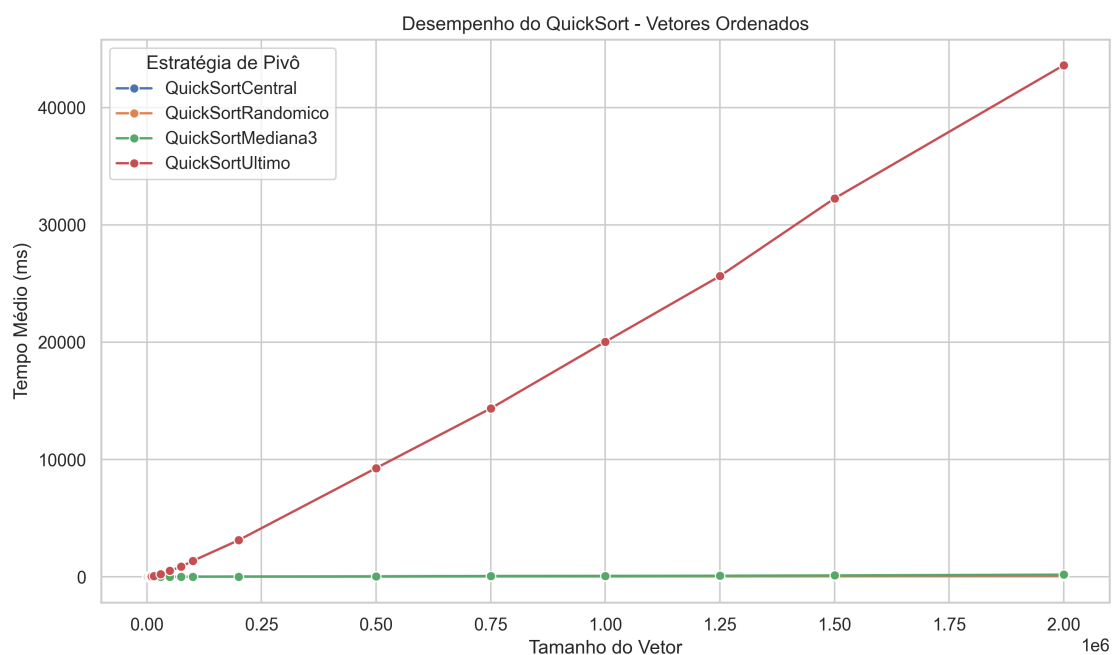


Figura 2. Efeito destrutivo do pior caso em arranjos previamente ordenados.

Para complementar a visualização gráfica do comportamento da estratégia do último elemento, a Tabela 2 apresenta os valores exatos em milissegundos extraídos das execuções. A discrepância numérica atesta que, enquanto os métodos mitigadores resolvem o problema em décimos de segundo, a abordagem ingênua atinge a marca de mais de 43.000 ms.

Tabela 2. Tempo médio de execução (em ms) para vetores Ordenados (Crescentes)

Tamanho	QS Central	QS Mediana 3	QS Randômico	QS Último
100	0.0012	0.0070	0.0628	0.0078
200	0.0036	0.0140	0.0094	0.0220
500	0.0086	0.0370	0.0236	0.1336
1.000	0.0468	0.0726	0.0742	0.6452
2.000	0.0712	0.1554	0.1066	1.9188
5.000	0.1536	0.4034	0.4570	10.1132
7.500	0.2848	0.6354	0.4554	21.0986
10.000	0.4238	0.9552	0.7174	39.0080
15.000	0.5508	1.3214	0.9844	72.3698
30.000	1.1568	2.7804	1.9512	218.0910
50.000	1.6720	3.8368	2.6494	504.9562
75.000	3.1790	6.9052	4.6646	873.7870
100.000	4.6966	9.7348	7.5368	1352.0254
200.000	8.8200	20.3224	13.6794	3118.9626
500.000	21.8462	44.2630	31.3722	9263.1424
750.000	33.1888	70.2130	45.0932	14351.6514
1.000.000	31.1784	78.4328	41.1808	20032.5460
1.250.000	44.8670	91.3960	58.0754	25632.6424
1.500.000	55.1770	117.8296	76.2338	32251.9532
2.000.000	65.4066	184.9672	102.9302	43609.5306

Decompondo a lógica da degradação devastadora do *QuickSortÚltimo*: ao acessar um vetor crescente e selecionar a última posição, ele adota como pivô o maior elemento da lista. O desmembramento joga todos os demais elementos à esquerda. O algoritmo itera repetidamente removendo apenas um único item por laço, resultando em um custo $\mathcal{O}(n^2)$ [Knuth 1998, Cormen et al. 2009].

Como constatado empiricamente pela Figura 2, a solução atingiu a marca superior a 40.000 milissegundos (43,6 segundos) de processamento. Em contrapartida, os métodos mitigadores (como o pivô Aleatório e Central) sequer despontam visualmente nestes gráficos, pois concluíram a mesma tarefa sob a fração temporal de cerca de 65 a 103 milissegundos, permanecendo achatados em suas bases $\mathcal{O}(n \log n)$.

4.3. Desempenho perante Dados Decrescentes

No cenário de vetores decrescentes, representado na Figura 3, verifica-se novamente a degradação quadrática da estratégia *QuickSortÚltimo*. Esse comportamento é similar ao observado no vetor ordenado, pois o último elemento de um vetor decrescente é sempre o menor valor, gerando igualmente o desbalanceamento máximo das partições.

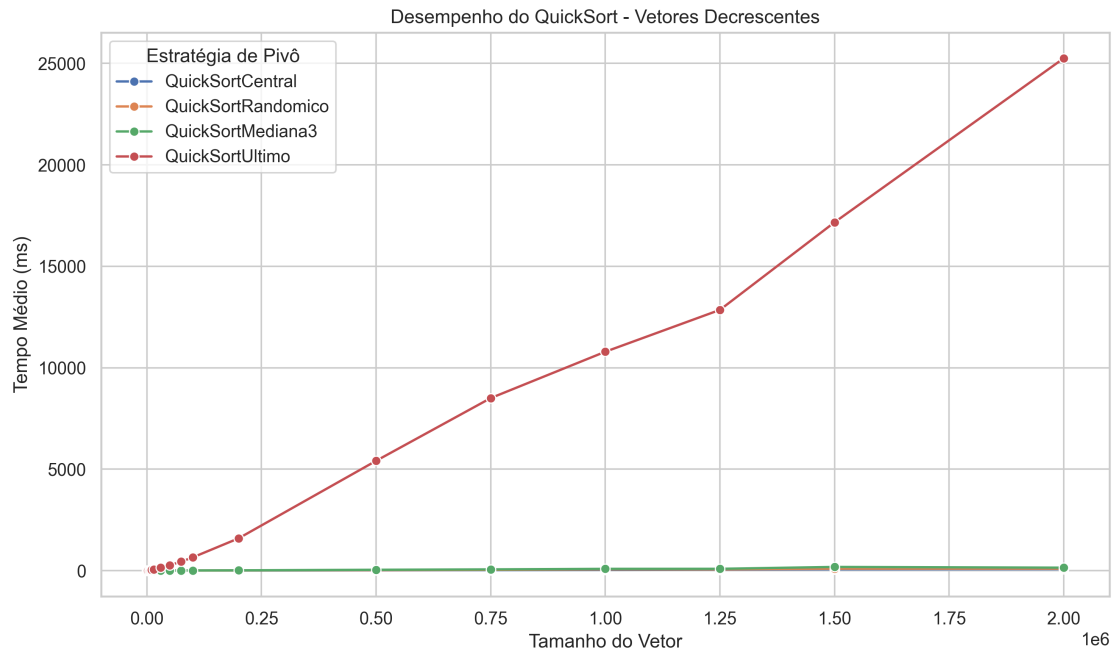


Figura 3. Degradação do método no processamento de vetores decrescentes.

Para embasar numericamente o que ocorreu no gráfico, a Tabela 3 expõe os tempos aferidos em milissegundos para vetores ordenados de forma invertida. Nota-se uma grande discrepância à medida que o vetor cresce: enquanto a estratégia de selecionar o último elemento penaliza a execução superando 25.000 ms no maior cenário, o particionamento Central lida de forma eficiente com o mesmo vetor em menos de 80 ms, corroborando as previsões matemáticas assintóticas.

Tabela 3. Tempo médio de execução (em ms) para vetores Decrescentes

Tamanho	QS Central	QS Mediana 3	QS Randômico	QS Último
100	0.0032	0.0260	0.0092	0.0232
200	0.0072	0.0220	0.0256	0.0414
500	0.0166	0.0664	0.0500	0.3860
1.000	0.0294	0.0946	0.0730	0.7780
2.000	0.0906	0.1946	0.1318	2.1778
5.000	0.4016	0.5002	0.3934	10.8648
7.500	0.3172	0.7656	0.6344	20.0384
10.000	0.5834	1.4376	1.2210	34.0268
15.000	0.7294	2.0278	1.3468	55.1418
30.000	1.0814	2.7944	2.6594	145.9544
50.000	1.8856	4.2092	2.9802	268.8480
75.000	2.7020	6.0294	4.2384	445.3058
100.000	3.7554	8.7104	5.8120	652.8524
200.000	7.5650	15.3152	11.4098	1591.0850
500.000	16.3136	42.9634	25.6764	5411.9350
750.000	28.3868	61.7998	40.7498	8503.3900
1.000.000	38.0108	88.7426	64.2892	10788.5626
1.250.000	48.6494	90.6176	62.0356	12851.4150
1.500.000	66.5674	182.9196	98.4228	17166.6134
2.000.000	75.2680	146.5590	98.8888	25251.3960

Neste contexto, a linha vermelha correspondente à estratégia *QuickSortUltimo* já demonstra uma degradação severa. Ao escolher sempre o último elemento em um vetor invertido, o pivô acaba sendo frequentemente o menor elemento da partição, o que impossibilita um balanceamento eficiente e distancia a curva do comportamento $\mathcal{O}(n \log n)$. Não obstante, as demais estratégias (Central, Randômica e Mediana de Três Aleatorizada) preservaram a estabilidade assintótica $\mathcal{O}(n \log n)$, mantendo tempos de execução extremamente reduzidos (75,27 ms, 146,56 ms e 98,89 ms, respectivamente).

4.4. Conjuntos Parcialmente Ordenados

No cenário de vetores parcialmente ordenados, ilustrado na Figura 4, a estratégia do *QuickSortUltimo* apresentou o maior tempo empírico absoluto registrado em todo o estudo, alcançando 44.899,89 ms ($\approx 44,9$ segundos), valor inclusive superior ao obtido com o vetor estritamente ordenado (43.609,53 ms). O alto grau de ordenação local presente no vetor faz com que a escolha do último elemento atinja repetidamente partições severamente desequilibradas, aproximando-se do pior caso de particionamento.

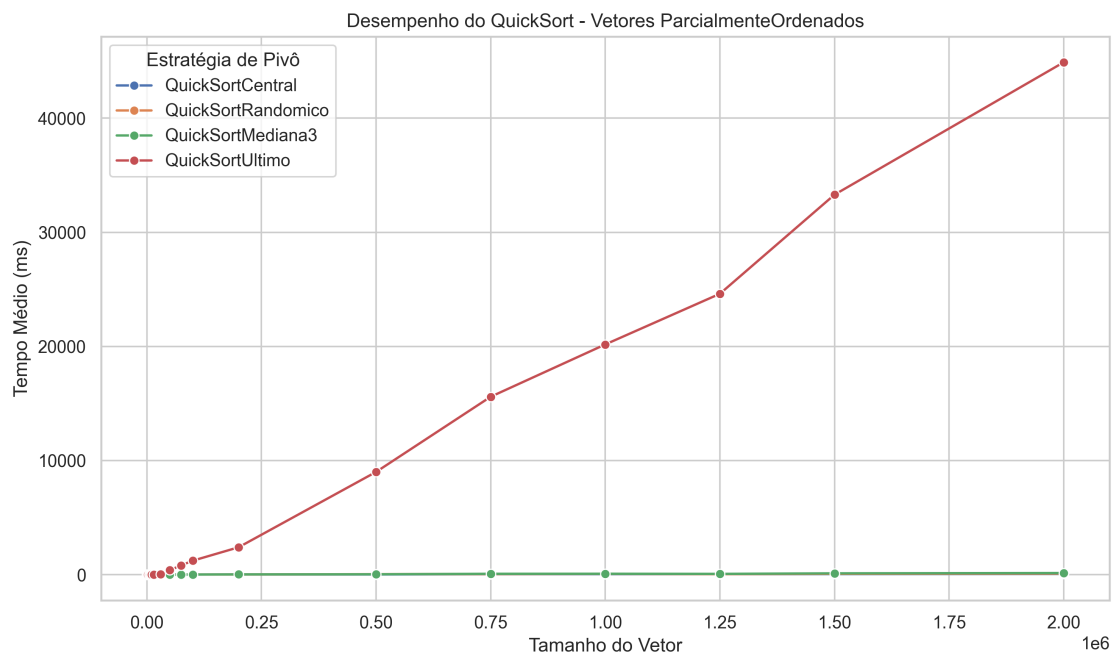


Figura 4. Processamento do QuickSort em vetores parcialmente ordenados.

Para viabilizar uma investigação quantitativa do fenômeno, a Tabela 4 condensa os milissegundos dispendidos no processamento das instâncias parcialmente ordenadas. A leitura direta dos dados numéricos evidencia que a presença de qualquer padrão orgânico na matriz já é gatilho suficiente para a degradação da estratégia do último elemento.

Tabela 4. Tempo médio de execução (em ms) para vetores Parcialmente Ordenados

Tamanho	QS Central	QS Mediana 3	QS Randômico	QS Último
100	0.0020	0.0114	0.0090	0.0064
200	0.0044	0.0216	0.0168	0.0086
500	0.0236	0.0792	0.0446	0.0956
1.000	0.0644	0.1240	0.0748	0.1300
2.000	0.1098	0.2654	0.2412	0.5672
5.000	0.3144	0.4436	0.3014	1.6984
7.500	0.3878	0.5822	0.5440	3.3052
10.000	0.5602	0.8804	0.6290	6.0954
15.000	0.5004	1.1830	0.8588	6.3758
30.000	1.0862	3.4120	2.2932	47.2064
50.000	2.6554	5.2902	3.8110	395.0352
75.000	2.7368	5.4604	3.7566	812.8202
100.000	5.0306	10.0098	7.8656	1222.1646
200.000	12.6384	21.3962	14.1692	2396.7242
500.000	18.2846	31.8438	41.4682	9001.5842
750.000	37.8848	76.1500	48.6834	15589.3630
1.000.000	48.3996	79.0372	70.1516	20173.8576
1.250.000	45.2296	69.1340	43.2236	24614.2522
1.500.000	56.2528	115.9134	75.8016	33297.1242
2.000.000	71.5272	142.7586	92.0730	44899.8878

A razão pela qual esse cenário superou até mesmo o vetor perfeitamente ordenado merece destaque analítico. No vetor estritamente ordenado, o pivô é *sempre* o maior elemento da partição corrente, gerando partições perfeitamente regulares de tamanhos 0 e $n - 1$, com custo de particionamento estritamente linear em cada nível de recursão [Hoare 1962, Cormen et al. 2009]. No vetor parcialmente ordenado, as perturbações pontuais fazem o pivô variar: ora ele é o maior elemento (partição $0/(n-1)$), ora é um elemento ligeiramente abaixo do máximo, gerando partições desequilibradas do tipo $k/(n-1-k)$ com k pequeno, porém não nulo. Essa irregularidade preserva a profundidade quase quadrática da recursão e, ao mesmo tempo, impede que o particionador termine precocemente, acrescentando trocas e comparações sem produzir divisões úteis [Knuth 1998]. O resultado é um número médio de operações por nível de recursão ligeiramente maior, explicando o custo final superior ao caso totalmente ordenado.

Em contrapartida, as demais abordagens demonstraram robustez notável. O *QuickSortCentral* atinge tempos brilhantes e é o mais veloz do conjunto geral, registrando marcas na casa dos 71,53 ms nestes arranjos. Isso se explica por conta de sua matemática de seleção não envolver *random*. Ele simplesmente calcula a divisão por 2 de dois inteiros e resgata um elemento da RAM. Quando os dados possuem ordenação aparente, o índice do meio frequentemente é o pivô aritmético exato do arranjo.

4.5. Comparativo Global (Escala Logarítmica)

A fim de encapsular todos os métodos perante todo o conjunto de dados, foi gerada uma plotagem em que a variação de tempo sofre aplicação de escala Logarítmica para encurtar a disparidade de 40.000 ms contra 70 ms em uma mesma prancheta. A Figura 5 encerra a análise de resultados.

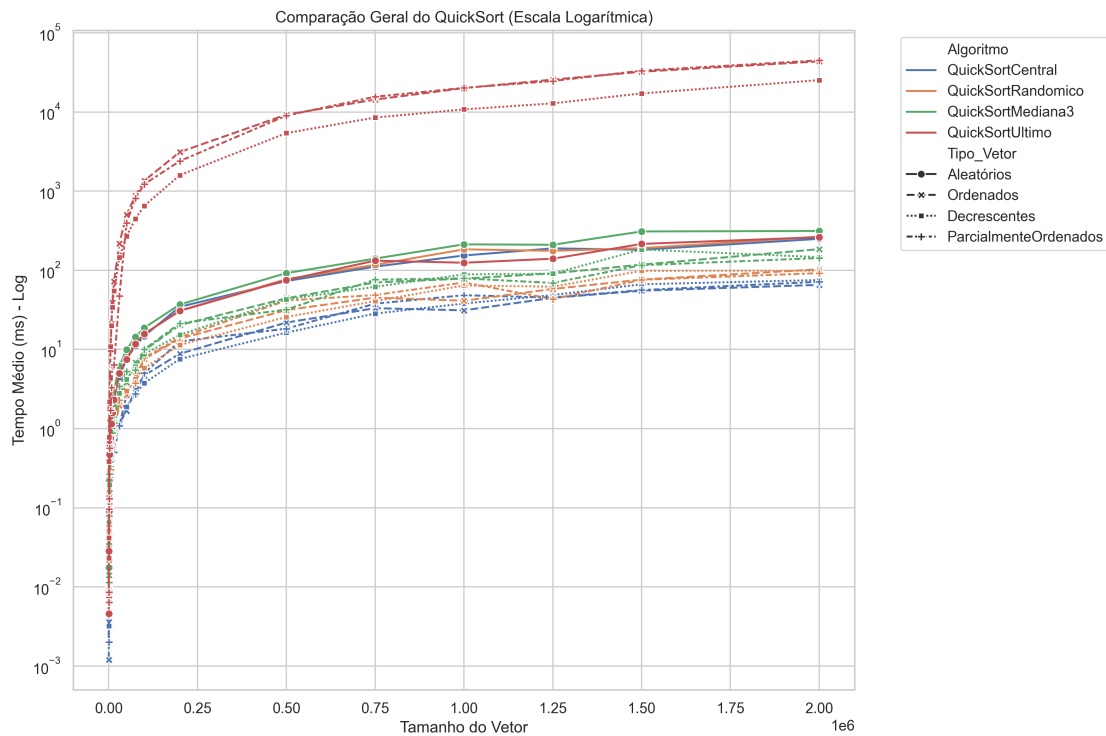


Figura 5. Comparação geral de tempo do QuickSort aplicando conversão logarítmica (ms).

Nela é possível enxergar, de forma imediata, a separação entre dois grupos distintos: as estratégias Central, Randômica e Mediana de Três Aleatorizada formam um feixe compacto na parte inferior, cujo crescimento acompanha o padrão linear-logarítmico $\mathcal{O}(n \log n)$ em todas as distribuições; já a curva do *QuickSortUltimo* permanece próxima ao grupo em vetores aleatórios, mas descola-se violentamente em direção à escala quadrática para entradas estruturadas, subindo duas potências logarítmicas de distância da base ($\mathcal{O}(n^2)$). Essa leitura global justifica a discussão cenário a cenário apresentada na sequência.

5. Conclusões Finais

A partir da análise teórica e empírica conduzida neste estudo, é possível estabelecer as seguintes conclusões fundamentais. Em primeiro lugar, a estratégia *QuickSortUltimo* é computacionalmente vulnerável e inapropriada para aplicações gerais, pois degrada para o pior caso quadrático $\mathcal{O}(n^2)$ sempre que encontra dados com ordenação prévia (crescente, decrescente ou parcial), apresentando um tempo de execução mais de 400 vezes superior às demais abordagens em grandes volumes de dados [Knuth 1998, Cormen et al. 2009].

Em segundo lugar, a estratégia *QuickSortCentral* apresentou o melhor desempenho prático global, combinando um baixíssimo *overhead* de computação do índice do pivô com particionamentos perfeitamente simétricos em entradas pré-ordenadas [Cormen et al. 2009]. Convém registrar, porém, que esse desempenho destacado em vetores ordenados decorre de uma coincidência favorável do cenário de teste: em vetores estritamente aleatórios, a posição central do subvetor não garante, em geral, um pivô próximo da mediana real, de modo que a estratégia Randômica permanece a recomendação mais segura para cargas de dados de natureza desconhecida [Sedgewick and Wayne 2011].

Em terceiro lugar, a estratégia *QuickSortRandomico* provou ser uma solução altamente consistente e imune a padrões de pré-ordenação nos dados de entrada, mantendo escala $\mathcal{O}(n \log n)$ com baixos tempos absolutos [Hoare 1962, Cormen et al. 2009].

Por último, a amostragem por Mediana de Três Aleatorizada assegura estabilidade assintótica [Sedgewick and Wayne 2011]; contudo, o custo adicional de três sorteios estocásticos e comparações condicionais por chamada recursiva torna essa estratégia ligeiramente mais lenta em termos de tempo de relógio do que a Central e a Randômica.

Este trabalho conclui que a seleção e o uso do QuickSort em *software* necessitam, obrigatoriamente, ser acompanhados da aplicação de Randomização ou Mediana se não for conhecida a estrutura dos dados a serem injetados. Em contrapartida, se houver segurança de que os dados carregam sempre ordenações orgânicas aparentes, o método do pivô Central demonstra o melhor custo-benefício computacional de processamento dentre as abordagens estudadas.

Referências

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, 3rd edition.
- Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1):10–16.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition.
- Sedgewick, R. and Wayne, K. (2011). *Algorithms*. Addison-Wesley Professional, 4th edition.